PYTHON — CAPITOLO 1

Fondamenti di Python

Teoria · Esempi · Errori comuni · Esercizi

1. Variabili e assegnazioni

Una variabile è un nome associato a un oggetto. L'operatore = assegna un oggetto a un nome.

```
X = 10
y = x
x = 20
print(x)  # 20
print(y)  # 10
```

Quando `y = x` viene eseguito, `y` si riferisce al valore che `x` aveva in quel momento, cioè `10`. In seguito, riassegnare `x` a `20` non cambia `y`.

Da ricordare: assegnare un valore a una variabile non significa necessariamente copiare l'oggetto.

1.1 Liste, riferimenti e mutabilità

Una lista è un oggetto mutabile: il suo contenuto può essere modificato dopo la creazione.

```
A = [1, 2, 3]
b = a
a.append(4)
print(a)  # [1, 2, 3, 4]
print(b)  # [1, 2, 3, 4]
```

`a` e `b` si riferiscono alla stessa lista. Il metodo `append(4)` modifica quella lista, quindi la modifica è visibile attraverso entrambi i nomi.

Consideriamo ora una riassegnazione:

```
a = [1, 2, 3]
b = a
a.append(4)
a = [7, 8, 9]
print(a)  # [7, 8, 9]
print(b)  # [1, 2, 3, 4]
```

La riga `a = [7, 8, 9]` assegna ad `a` una nuova lista. Non modifica la lista precedente, a cui `b` continua a riferirsi.

Differenza fondamentale:

- `a.append(4)` modifica la lista esistente.
- `a = [7, 8, 9]` assegna ad `a` una nuova lista.

1.2 Copie superficiali

Il metodo `copy()` crea una copia superficiale (shallow copy) di una lista.

```
a = [[1, 2], [3, 4]]
b = a.copy()
a[0].append(5)
print(b)  # [[1, 2, 5], [3, 4]]
```

Perché `b` cambia?

`a.copy()` crea una nuova lista esterna, ma gli elementi annidati continuano a essere condivisi. In questo caso, `a[0]` e `b[0]` si riferiscono alla stessa lista interna.

Una copia superficiale, quindi, non copia ricorsivamente tutti gli oggetti contenuti.

2. Funzioni: parametri, `print()` e `return`

Una funzione raggruppa istruzioni che possono essere eseguite quando la funzione viene chiamata.

```
def somma(a, b):
      return a + b
risultato = somma(3, 4)
print(risultato)  # 7
```

In questo esempio:

- `a` e `b` sono parametri.
- `3` e `4` sono gli argomenti passati alla funzione.
- `return a + b` restituisce il risultato della somma.
- `risultato` riceve il valore restituito, cioè `7`.

2.1 Differenza tra `print()` e `return`

Questa è una delle distinzioni più importanti da comprendere.

`print()` mostra un valore sullo schermo.

`return` restituisce un valore al punto in cui la funzione è stata chiamata.

Esempio con `print()`:

```
def somma(a, b):
      print(a + b)
risultato = somma(3, 4)
print(risultato)
```

Output:

```
7
None
```

La funzione stampa `7`, ma non contiene un `return` esplicito. Quando termina, restituisce automaticamente `None`.

Di conseguenza, `risultato` contiene `None`.

Esempio con `return`:

```
def somma(a, b):
      return a + b
risultato = somma(3, 4)
print(risultato)
```

Output:

```
7
```

Questa volta la funzione restituisce `7`, che viene assegnato a `risultato`.

2.2 Che cos'è `None`?

`None` è un oggetto speciale di Python che rappresenta l'assenza di un valore significativo.

```
def esempio():
      print("Ciao")
risultato = esempio()
print(risultato)
```

Output:

```
Ciao
None
```

`None` non è né `0` né `False`, anche se in alcuni contesti può essere valutato come falso. È un valore distinto.

2.3 `return` interrompe la funzione

Quando Python esegue un `return`, restituisce il valore e termina immediatamente la funzione nel percorso di esecuzione corrente.

```
def esempio():
      return 5
      print("Ciao")
esempio()
```

In questo caso:

- La funzione restituisce `5`.
- `print("Ciao")` non viene eseguito.
- Il valore `5` non viene stampato automaticamente, perché non è stato usato `print()`.

Per visualizzare il valore restituito:

```
print(esempio())  # 5
```

2.4 Riutilizzare i valori restituiti

Il valore restituito da una funzione può essere passato a un'altra chiamata.

```
def raddoppia(numero):
      return numero * 2
```

```
x = raddoppia(5)
y = raddoppia(x)
print(x)  # 10
print(y)  # 20
```

Prima `raddoppia(5)` restituisce `10`. Poi `raddoppia(x)` riceve `10` e restituisce `20`.

3. Scope: variabili locali e globali

Lo scope è l'ambito nel quale un nome è accessibile.

Una variabile definita fuori da una funzione è spesso chiamata globale. Una variabile definita dentro una funzione è normalmente locale a quella funzione.

3.1 Variabili locali

```
Numero = 10
      def modifica():
      numero = 5
      print(numero)
modifica()
print(numero)
```

Output:

```
5
10
```

La funzione crea una variabile locale `numero`, distinta dalla variabile globale con lo stesso nome.

L'assegnazione locale non modifica quella globale.

3.2 Leggere una variabile globale

Una funzione può leggere una variabile globale se non esiste un'assegnazione locale che faccia riferimento allo stesso nome.

```
Numero = 10
def mostra():
      print(numero)
mostra()
```

Output:

```
10
```

La funzione legge il valore della variabile globale.

3.3 Perché alcune assegnazioni generano errori?

```
Numero = 10
def modifica():
      numero = numero + 5
print(numero)
modifica()
```

Questo codice genera un `UnboundLocalError`.

Poiché dentro la funzione viene assegnato un valore a `numero`, Python considera quel nome locale nella funzione. L'espressione cerca però di leggere il valore locale prima che sia stato inizializzato.

3.4 Passare valori attraverso i parametri

Un modo chiaro per lavorare con i valori è passarli come argomenti.

```
def modifica(valore):
      valore = valore + 5
      return valore
numero = 10
risultato = modifica(numero)
print(numero)     # 10
print(risultato)  # 15
```

La funzione riceve `10`, calcola `15` e lo restituisce. La variabile globale `numero` rimane `10`.

Buona pratica: preferisci parametri e valori restituiti quando vuoi rendere il comportamento di una funzione chiaro e prevedibile.

4. Booleani e confronti

I valori booleani sono:

- `True`: vero.
- `False`: falso.

Le espressioni di confronto restituiscono direttamente un valore booleano.

Operatore	Significato	Esempio
`==`	Uguale a	`5 == 5` → `True`
`!=`	Diverso da	`5 != 3` → `True`
`>`	Strettamente maggiore	`10 > 10` → `False`
`>=`	Maggiore o uguale	`10 >= 10` → `True`
`<`	Strettamente minore	`7 < 10` → `True`
`<=`	Minore o uguale	`10 <= 10` → `True`

Attenzione alla differenza:

- `>` esclude l'uguaglianza.
- `>=` include l'uguaglianza.

4.1 Restituire direttamente un confronto

Una funzione può restituire direttamente un'espressione che produce `True` o `False`.

```
def è_maggiore_di_dieci(numero):
      return numero > 10
print(è_maggiore_di_dieci(15))  # True
print(è_maggiore_di_dieci(7))   # False
```

Non è necessario usare `if` e `else` per restituire esplicitamente `True` oppure `False` in questo caso.

5. Operatori logici

Gli operatori logici permettono di combinare condizioni.

5.1 and

`and` restituisce un risultato vero soltanto quando entrambe le condizioni sono vere.

```
numero >= 0 and numero <= 100
```
Questa condizione è vera se `numero` è compreso tra 0 e 100, estremi inclusi.

5.2 or

`or` restituisce un risultato vero se almeno una delle condizioni è vera.

```
numero < 0 or numero > 100
```
Questa condizione è vera se il numero è minore di 0 oppure maggiore di 100.

5.3 not

`not` inverte un valore booleano:

```
not True   # False
not False  # True
```

5.4 Combinare and e or

Considera:

```
numero >= 0 and numero <= 100 or numero == -1
```
In Python, `and` ha precedenza su `or`. L'espressione equivale quindi a:

```
(numero >= 0 and numero <= 100) or (numero == -1)
```
Le parentesi sono utili per rendere chiaro il raggruppamento delle condizioni.

6. Condizioni: `if`, `elif`, `else`

Le condizioni permettono di scegliere quale blocco di codice eseguire.

```
def descrivi(numero):
      if numero < 0:
            return "negativo"
      elif numero == 0:
            return "zero"
      else:
            return "positivo"
```

- `if`: controlla la prima condizione.
- `elif`: controlla una condizione aggiuntiva se quelle precedenti sono false.

- `else`: viene eseguito quando nessuna delle condizioni precedenti è vera.

Esempio:

```
print(descrivi(-3))  # negativo
print(descrivi(0))   # zero
print(descrivi(8))   # positivo
```

Una volta eseguito un ramo che contiene `return`, la funzione termina e non controlla gli altri rami.

7. Anteprima dei cicli: `for` e `range()`

I cicli permettono di ripetere un blocco di codice.

```
for numero in [1, 2, 3]:
      print(numero * 2)
```

Output:

```
2
4
6
```

Il blocco viene eseguito una volta per ogni elemento della lista.

La funzione `range()` permette di generare una sequenza di numeri.

```
for numero in range(1, 5):
      print(numero)
```

Output:

```
1
2
3
4
```

`range(1, 5)` parte da `1` e si ferma prima di `5`: il limite finale è escluso.

7.1 Sommare i valori di un ciclo

```
Totale = 0
for numero in range(1, 5):
      totale = totale + numero
print(totale)
```

Il ciclo aggiunge a `totale` i valori `1`, `2`, `3` e `4`. Il risultato finale è `10`.

Questo argomento verrà approfondito nel Capitolo 2.

8. Errori comuni da evitare

1. Confondere `print()` con `return`. Il primo mostra un valore; il secondo lo restituisce.
2. Aspettarsi che una funzione senza `return` restituisca il valore stampato. Restituisce `None`.

3. Confondere la mutazione con la riassegnazione. Modificare una lista non è la stessa cosa che assegnare una nuova lista a un nome.
4. Pensare che una variabile locale modifichi automaticamente quella globale. Sono nomi distinti in ambiti diversi.
5. Dimenticare che `return` termina la funzione. Le istruzioni successive non vengono eseguite in quel percorso.
6. Confondere `and` e `or`. Con `and` devono essere vere entrambe le condizioni; con `or` ne basta una.
7. Dimenticare che `range()` esclude il limite finale. `range(1, 5)` produce `1`, `2`, `3`, `4`.

9. Esercizi di ripasso

Prova a risolverli senza guardare le soluzioni.

Esercizio 1 — Riferimenti

```
A = [1, 2]
b = a
a.append(3)
a = [9, 9]
print(a)
print(b)
```

Esercizio 2 — `print()` e `return`

```
def calcola(x):
      print(x * 2)
      return x + 1
risultato = calcola(4)
print(risultato)
```

Esercizio 3 — Scope

```
Numero = 3
def calcola(numero):
      numero = numero * 2
      return numero + 1
risultato = calcola(numero)
print(numero)
print(risultato)
```

Esercizio 4 — Condizioni

```
def controlla(n):
      return n >= 0 and n < 10 or n == -1
print(controlla(0))
print(controlla(10))
print(controlla(-1))
print(controlla(-5))
```

Soluzioni ragionate

Esercizio 1

```
[9, 9]
[1, 2, 3]
```

`append()` modifica la lista condivisa; la riassegnazione crea una nuova lista per `a`.

Esercizio 2

```
8
5
```

La funzione stampa `8` e restituisce `5`, che viene assegnato a `risultato`.

Esercizio 3

```
3
7
```

Il parametro locale viene moltiplicato per due e poi incrementato. La variabile globale rimane `3`.

Esercizio 4

```
True
False
True
False
```

La condizione accetta i numeri da 0 a 9 inclusi e anche `-1`.

10. Checklist di fine capitolo

Prima di passare al capitolo successivo, dovresti riuscire a:

- Distinguere tra riassegnazione e modifica di una lista.
- Spiegare perché due nomi possono riferirsi allo stesso oggetto.
- Distinguere `print()` da `return`.
- Spiegare che cosa rappresenta `None`.
- Seguire l'ordine di esecuzione di una funzione.
- Distinguere variabili locali e globali.
- Prevedere i risultati di confronti e operatori logici.
- Leggere condizioni `if`, `elif` ed `else`.
- Comprendere che `range()` esclude il limite finale.

Capitolo 1 completato.

Per esportare questi appunti in PDF da Ubuntu, puoi copiare il testo in LibreOffice Writer, applicare gli stili dei titoli e scegliere File → Esporta come → Esporta direttamente come PDF.

Nel prossimo capitolo approfondiremo i cicli `for` e `while`, `range()`, `break`, `continue` e i cicli annidati.